

Tarea 1: Stack, Queue y Dictionary

Alumno: Fernando Renteria A00836509

Fecha: 1 de octubre de 2026

Repositorio: <https://github.com/FernandoRent/Tarea1>

1. Qué hice

Hice tres clases en C++17. Son plantillas, viven solo en headers y están dentro del namespace `structures`:

- `Stack<T>`: pila LIFO. Por dentro usa un `std::vector` y el tope es el final.
- `Queue<T>`: fila FIFO. Usa un `std::deque`, así que meter por atrás y sacar por adelante es $O(1)$.
- `Dictionary<K, V>`: guarda pares llave-valor y recuerda el orden en que se insertaron. Usa una `std::list` para el orden y un `std::unordered_map` que lleva de cada llave a su nodo en la lista. Con eso buscar, insertar y borrar cuesta $O(1)$ en promedio.

Las tres tienen las operaciones de siempre: `push`, `pop` y `peek`; `enqueue`, `dequeue`, `front` y `back`; y `put`, `get`, `getOr`, `remove`, `contains`, `keys`, `values`, `items`, `sortedItems` y el operador `[]`. También se pueden recorrer con un `for` y se imprimen con `<<`. Aceptan valores por copia o por movimiento, así que funcionan hasta con `std::unique_ptr`. Si se intenta algo inválido, como hacer `pop` en una pila vacía, lanzan una excepción propia (`EmptyStackException`, `EmptyQueueException` o `KeyNotFoundException`); las tres heredan de `StructureException`. Cada método está documentado con su complejidad.

En `src/main.cpp` hay un demo que usa las tres con ejemplos del compilador del curso: evalúa "`3 4 + 2 *`" con una pila de operandos, procesa una fila de cuádruplos y arma una tabla de símbolos.

2. ¿Desde cero, con librerías o con IA?

Desde cero

No del todo. Las estructuras se apoyan en contenedores de la biblioteca estándar de C++ (`vector`, `deque`, `list` y `unordered_map`), algo que el enunciado permite. Lo propio es lo que se arma encima: la interfaz, las validaciones, las excepciones y, en el diccionario, juntar la lista con el hash para tener orden y búsqueda rápida a la vez.

Librerías

Solo la biblioteca estándar de C++. Para las pruebas usé `GoogleTest`, que `CMake` descarga solo. El proyecto se compila con `CMake` sin warnings (`-Wall -Wextra -Wpedantic`).

Inteligencia artificial

Sí, usé Claude dentro de VS Code. Yo definí lo que quería y le pedí que me guiara en lugar de darme el código hecho. Así hice `Exceptions.hpp`, `Stack.hpp`, `Queue.hpp` y la base del `CMakeLists.txt`: los escribí yo, y Claude me explicaba los conceptos y me revisaba los errores. En `Stack` y `Queue` también me ayudó con los comentarios de documentación.

Después le pedí que escribiera `Dictionary.hpp`, los tres archivos de pruebas, y la configuración de `GoogleTest`, y que me explicara cómo funcionan. La conversación completa va en el

zip (Conversacion_Tarea1.md).

3. Test-cases

Hice 55 pruebas con GoogleTest, que se corren con ctest, y pasaron todas. Cada prueba lleva su ID en el nombre (por ejemplo `S03_PopReturnsInLifoOrder`), y el detalle de cada una (pasos, resultado esperado y obtenido) está en `TEST_CASES.txt`. Revisan el uso normal, los casos borde (estructura vacía, un solo elemento, 10,000 elementos, `std::string` y `std::unique_ptr`) y los casos de error, donde se verifica que salga la excepción correcta.

Stack (15 pruebas)

ID	Qué revisa
S01	Una pila nueva está vacía.
S02	peek muestra el último que entró.
S03	pop saca en orden inverso: 3, 2, 1.
S04	pop en pila vacía lanza <code>EmptyStackException</code> .
S05	peek en pila vacía también lanza (versión normal y const).
S06	Con un solo elemento: push, peek y pop; luego queda vacía.
S07	Se puede cambiar el tope a través de peek.
S08	peek funciona en una pila const.
S09	clear vacía la pila, incluso si ya estaba vacía.
S10	El for recorre del tope al fondo.
S11	Se imprime como [], [7] y [9, 8, 7].
S12	Con strings, push por copia no cambia la variable original.
S13	Funciona con <code>std::unique_ptr</code> (solo se puede mover).
S14	Aguanta 10,000 elementos y los saca en orden.
S15	La excepción también se atrapa como <code>StructureException</code> y <code>runtime_error</code> , con su mensaje.

Queue (16 pruebas)

ID	Qué revisa
Q01	Una fila nueva está vacía.
Q02	front es el primero que entró y back el último. Q03 dequeue saca en el mismo orden: 1, 2, 3.
Q04	dequeue en fila vacía lanza <code>EmptyQueueException</code> .
Q05	front en fila vacía lanza (normal y const).
Q06	back en fila vacía lanza (normal y const).
Q07	Con un elemento, front y back son el mismo.
Q08	Se pueden cambiar el primero y el último a través de front y back.
Q09	clear vacía la fila.
Q10	El for recorre del frente al final.
Q11	Se imprime como [], [7] y [7, 8, 9].
Q12	Con strings, enqueue por copia no cambia la variable original.
Q13	Funciona con <code>std::unique_ptr</code> .
Q14	Aguanta 10,000 elementos y los saca en orden.

ID	Qué revisa
Q15	Meter y sacar intercalado sigue respetando FIFO.
Q16	La excepción se atrapa como StructureException y trae su mensaje.

Dictionary (24 pruebas)

ID	Qué revisa
D01	Un diccionario nuevo está vacío.
D02	Lo que se guarda con put se recupera con get.
D03	Repetir una llave actualiza el valor, no la duplica.
D04	Actualizar una llave no le cambia su lugar en el orden.
D05	get de una llave que no existe lanza KeyNotFoundException (normal y const).
D06	Se puede cambiar un valor a través de get.
D07	getOr regresa el valor por defecto y no inserta nada.
D08	contains se actualiza después de put y remove.
D09	remove regresa true si la llave existía y false si no.
D10	Si borras una llave y la vuelves a meter, queda al final.
D11	El operador [] con una llave nueva la crea con valor por defecto.
D12	El operador [] con una llave existente no inserta otra.
D13	keys, values e items salen en orden de inserción.
D14	sortedItems ordena por llave sin desordenar el diccionario.
D15	El for recorre en orden de inserción y puede cambiar valores.
D16	Se imprime como {}, {x: 1} y {x: 1, y: 2}.
D17	clear lo deja vacío por completo.
D18	Funciona con un solo elemento (llave int, valor string).
D19	Una copia es independiente del original.
D20	Asignar por copia también es independiente, y asignarse a sí mismo no rompe nada.
D21	Al moverlo, el original queda vacío y el nuevo funciona.
D22	Acepta valores std::unique_ptr.
D23	10,000 inserciones y 5,000 borrados, y el orden se mantiene.
D24	La excepción se atrapa como StructureException y trae su mensaje.

4. Cómo correrlo

Desde la carpeta del proyecto:

```
cmake -S . -B build -G "MinGW Makefiles"
cmake --build build
.\build\demo.exe
ctest --test-dir build --output-on-failure
```

El primero configura, el segundo compila, el tercero corre el demo y el último corre las pruebas.